

Recurrent Neural Networks

LSTM and Gated RNNs

Introduction to Deep Learning · Class 28

Outline

Why a Simple Cell Forgets

Paths Through Time

The LSTM

Other Gated Cells

Optimising Through Time

Putting It Together

Why a Simple Cell Forgets

Opening the cell

Class 27 used the recurrent cell g as a black box. Today the box is opened: why $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ cannot hold a fact for long, and what a gate changes.

- *The flights the airline was canceling **were** full.* To choose *were*, the model must carry the plural *flights* across *airline was*, which argues for the singular
- The hidden state has two jobs at once: serve the current decision, and carry what future decisions will need
- And the gradient that would teach it the second job passes through as many multiplications as there are steps between cause and effect

Both problems have the same root: the state is overwritten at every step by the same matrix. A gate lets the network decide, at each step, what to overwrite.

Jurafsky & Martin (2026), §14.5, example (14.19); Goodfellow et al. (2016), §10.7.

Notation for today

$\mathbf{h}_t, \mathbf{c}_t$	hidden state and cell (context) state	$\odot$	element-wise product
$\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t$	forget, add (input) and output gates	σ	the logistic sigmoid
$\mathbf{g}_t$	the candidate written into the cell	λ_i	eigenvalues of $\mathbf{U}$, as in Class 13
$\mathbf{u}_t, \mathbf{r}_t$	the GRU's update and reset gates	$\mathbf{J}_t$	$\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}$, the step Jacobian

- Jurafsky's letters throughout: $\mathbf{U}$ recurrent, $\mathbf{W}$ input, each gate with its own pair. Goodfellow swaps $\mathbf{U}$ and $\mathbf{W}$, writes the cell $\mathbf{s}$, the input gate $\mathbf{g}$ and the output gate $\mathbf{q}$
- Every equation from Goodfellow below is transcribed into these letters, with his number beside it

Jurafsky & Martin (2026), Eqs. 14.20–14.27; Goodfellow et al. (2016), Eqs. 10.40–10.47.

The linear case: a power iteration

Theorem 1 (Vanishing and exploding, without a nonlinearity)

For the recurrence $\mathbf{h}_t = \mathbf{U}\mathbf{h}_{t-1}$ with $\mathbf{U} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ orthogonally diagonalisable,

$$\mathbf{h}_t = \mathbf{U}^t \mathbf{h}_0 = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top \mathbf{h}_0,$$

so the component of $\mathbf{h}_0$ along eigenvector i is multiplied by λ_i^t : geometric decay if $|\lambda_i| < 1$, explosion if $|\lambda_i| > 1$; only the largest survives.

- **Proof.** $(\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top)^t = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top$ since $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, and $\mathbf{\Lambda}^t$ has entries λ_i^t . □
- The power method of Class 13, on the recurrent matrix instead of the Hessian. A feedforward net multiplies *different* matrices, which can be scaled to compensate; a recurrent net multiplies the *same* one.

Goodfellow et al. (2016), §10.7, Eqs. 10.36–10.39; Class 13.

The linear case: a power iteration

Theorem 1 (Vanishing and exploding, without a nonlinearity)

For the recurrence $\mathbf{h}_t = \mathbf{U}\mathbf{h}_{t-1}$ with $\mathbf{U} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ orthogonally diagonalisable,

$$\mathbf{h}_t = \mathbf{U}^t \mathbf{h}_0 = \mathbf{Q}\mathbf{\Lambda}^t \mathbf{Q}^\top \mathbf{h}_0,$$

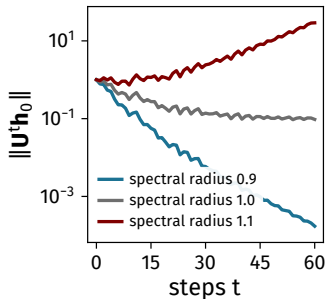
so the component of $\mathbf{h}_0$ along eigenvector i is multiplied by λ_i^t : geometric decay if $|\lambda_i| < 1$, explosion if $|\lambda_i| > 1$; only the largest survives.

- Proof. $(\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top)^t = \mathbf{Q}\mathbf{\Lambda}^t\mathbf{Q}^\top$ since $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$, and $\mathbf{\Lambda}^t$ has entries λ_i^t . □
- The power method of Class 13, on the recurrent matrix instead of the Hessian. A feedforward net multiplies *different* matrices, which can be scaled to compensate; a recurrent net multiplies the *same* one.

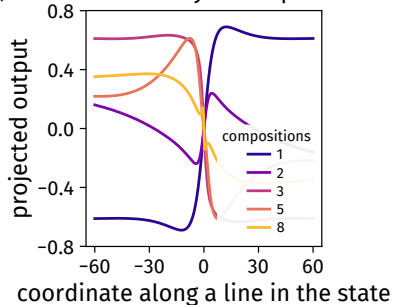
Goodfellow et al. (2016), §10.7, Eqs. 10.36–10.39; Class 13.

The power method, drawn

(a) the power method, three radii



(b) a linear--tanh layer composed t times



(a) $\|U^t h_0\|$ for a random 20×20 matrix rescaled to spectral radius 0.9, 1.0 and 1.1: after 60 steps the norm is 2×10^{-4} , 0.1 and 29. (b) a random linear-tanh layer composed 1 to 8 times, seen along one line through a 100-dimensional state: flat almost everywhere, steep in a few places, and more so with every composition. Goodfellow et al. (2016), Eqs. 10.36–10.39, Fig. 10.15.

With the nonlinearity: a bound on the gradient

Proposition 2 (Sufficient condition for vanishing)

For $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ with $|g'| \leq \gamma$ everywhere ($\gamma = 1$ for $\tanh$, $\frac{1}{4}$ for the sigmoid),

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \left\| \prod_{s=k+1}^t \mathbf{J}_s \right\| \leq (\gamma \|\mathbf{U}\|)^{t-k},$$

so if $\|\mathbf{U}\| < 1/\gamma$ the gradient from t back to k vanishes geometrically in the lag.

- **Proof.** $\mathbf{J}_s = \text{diag}(g') \mathbf{U}$ by the chain rule of Class 26, so $\|\mathbf{J}_s\| \leq \gamma \|\mathbf{U}\|$; the norm of a product is at most the product of the norms. □
- Sufficient, not necessary: it says when the gradient *must* vanish. Either way a distant cause gets an exponentially smaller gradient than a recent one, so the recent ones win.

Pascanu, Mikolov & Bengio (2013); Goodfellow et al. (2016), §10.7; Bengio et al. (1994).

With the nonlinearity: a bound on the gradient

Proposition 2 (Sufficient condition for vanishing)

For $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$ with $|g'| \leq \gamma$ everywhere ($\gamma = 1$ for $\tanh$, $\frac{1}{4}$ for the sigmoid),

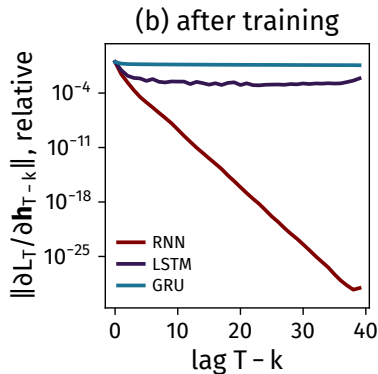
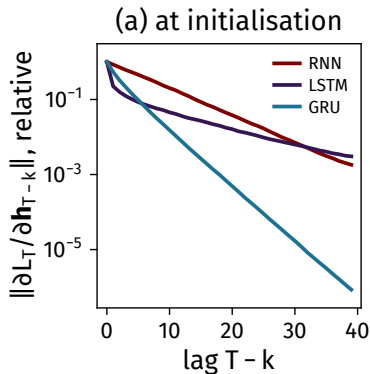
$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| = \left\| \prod_{s=k+1}^t \mathbf{J}_s \right\| \leq (\gamma \|\mathbf{U}\|)^{t-k},$$

so if $\|\mathbf{U}\| < 1/\gamma$ the gradient from t back to k vanishes geometrically in the lag.

- Proof. $\mathbf{J}_s = \text{diag}(g') \mathbf{U}$ by the chain rule of Class 26, so $\|\mathbf{J}_s\| \leq \gamma \|\mathbf{U}\|$; the norm of a product is at most the product of the norms. $\square$
- Sufficient, not necessary: it says when the gradient *must* vanish. Either way a distant cause gets an exponentially smaller gradient than a recent one, so the recent ones win.

Pascanu, Mikolov & Bengio (2013); Goodfellow et al. (2016), §10.7; Bengio et al. (1994).

The gradient by lag, measured



$\|\partial L_T / \partial \mathbf{h}_{T-k}\|$ against the lag k on length-40 sequences, read off the tape, for three cells of width 32 trained here to recall the first symbol. (a) at initialisation all three fall. (b) after training the simple RNN's gradient at the first step is 10^{-30} of its value at the last, the LSTM's 10^{-2} , the GRU's 0.4: the gated cells have learned to carry it.

Goodfellow et al. (2016), §10.7; Jurafsky & Martin (2026), §14.5.

Paths Through Time

Three ways to a path the gradient survives

Idea	Mechanism	Cost
skip connections	an edge from $t - d$ to t ; decay in τ/d	d fixed and integer
leaky units	a linear self-connection, weight $\alpha \approx 1$	one time scale per unit
removing connections	some units updated every d steps	groups and rates by hand
gates	a self-connection weighted by the input	more parameters; this class

- All four build a path along which the product of derivatives stays near one, so a fact can reach the present from far back
- Echo state networks give up instead: fix $\mathbf{U}$ at a chosen spectral radius (near, or above, one, with $\tanh$ to bound it) and learn only the output weights, a convex problem

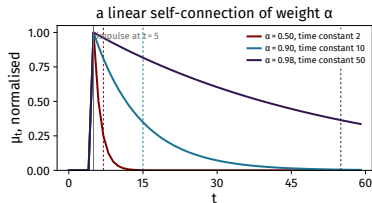
Goodfellow et al. (2016), §10.8–10.9; Lin et al. (1996); Mozer (1992); Jaeger & Haas (2004).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- **Proof. Substitute the recurrence into itself; the weights sum to one.** $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



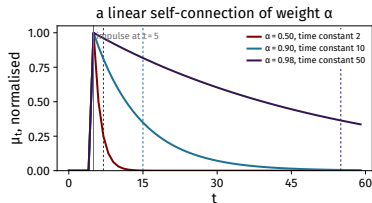
Goodfellow et al. (2016), §10.9.2; Mozer (1992).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- Proof. Substitute the recurrence into itself; the weights sum to one. $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



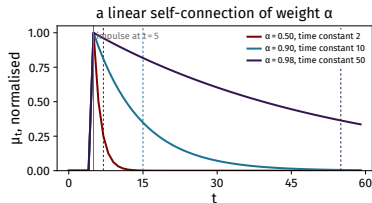
Goodfellow et al. (2016), §10.9.2; Mozer (1992).

The leaky unit is a running average

Proposition 3 (Time constant of a leaky unit)

The unit $\mu_t = \alpha\mu_{t-1} + (1 - \alpha)v_t$ unrolls to $\mu_t = (1 - \alpha) \sum_{s \leq t} \alpha^{t-s} v_s$: geometric weights with time constant $1/(1 - \alpha)$ steps.

- Proof. Substitute the recurrence into itself; the weights sum to one. $\square$
- Along the self-connection $\partial\mu_t/\partial\mu_{t-1} = \alpha$, a constant near one: the path the gradient survives.
- A gate replaces the constant α by a number in $(0, 1)$ computed at each step.



Goodfellow et al. (2016), §10.9.2; Mozer (1992).

The LSTM

Two problems, one design

- Split the state in two: a **cell** $\mathbf{c}_t$ that carries context, and a hidden state $\mathbf{h}_t$ that serves the current output. The two jobs get two vectors
- Manage the cell explicitly: **remove** what is no longer needed, **add** what later decisions will need, and let the network learn when to do each
- Every gate has the same design: a feedforward layer, a sigmoid, a pointwise product with the thing gated. The sigmoid pushes towards 0 or 1, so the product acts as a **soft mask**
- The cell's own update is additive, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (\dots)$, which is the leaky unit with α replaced by a gate

Jurafsky & Martin (2026), §14.5; Hochreiter & Schmidhuber (1997); Gers et al. (2000).

The LSTM cell, in equations

Definition 4 (LSTM)

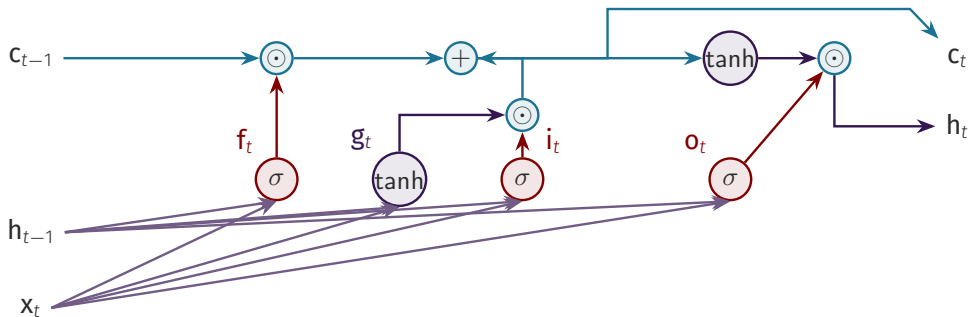
$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$	<i>forget gate</i>	$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$	<i>what survives</i>
$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$	<i>candidate</i>	$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$	<i>add gate</i>
$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$	<i>what is added</i>	$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$	<i>new cell</i>
$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$	<i>output gate</i>	$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$	<i>new hidden</i>

- Four weight pairs instead of one, for the same width. Goodfellow's Eqs. 10.40–10.44 are the same in his letters; his advice, after Gers et al., is a forget bias of 1 so the cell starts by remembering

Jurafsky & Martin (2026), Eqs. 14.20–14.27; Goodfellow et al. (2016), Eqs. 10.40–10.44.

The LSTM cell, drawn

top line: the cell is multiplied by f_t , added to, and passed on — no weight matrix on this path



Inputs c_{t-1} , h_{t-1} , x_t ; outputs c_t , h_t . Gates in maroon, activations in purple, elementwise operations in teal. From outside, the only difference from a simple cell is the extra vector in and out.

Jurafsky & Martin (2026), Fig. 14.13; Goodfellow et al. (2016), Fig. 10.16.

Why the cell remembers

Proposition 5 (The cell's gradient is a product of forget gates)

Holding the gates fixed, along the cell's own path $\partial \mathbf{c}_t / \partial \mathbf{c}_k = \prod_{s=k+1}^t \text{diag}(\mathbf{f}_s)$, a product of numbers in $(0, 1)$ with no weight matrix and no activation derivative in it: with $\mathbf{f}_s \approx \mathbf{1}$ the gradient passes unattenuated.

- **Proof.** $\mathbf{c}_s = \mathbf{f}_s \odot \mathbf{c}_{s-1} + \mathbf{j}_s$ is linear in $\mathbf{c}_{s-1}$ with coefficient $\mathbf{f}_s$; chain the steps. □
- Against Proposition 2: the simple cell's factor $\text{diag}(g') \mathbf{U}$ is typically well below one; the LSTM's is a gate the network can hold at one — the **constant error carousel**. The gates depend on $\mathbf{h}_{t-1}$ too, so there are other paths; this is the one that survives.

Hochreiter & Schmidhuber (1997); Gers et al. (2000); Goodfellow et al. (2016), §10.10.1.

Why the cell remembers

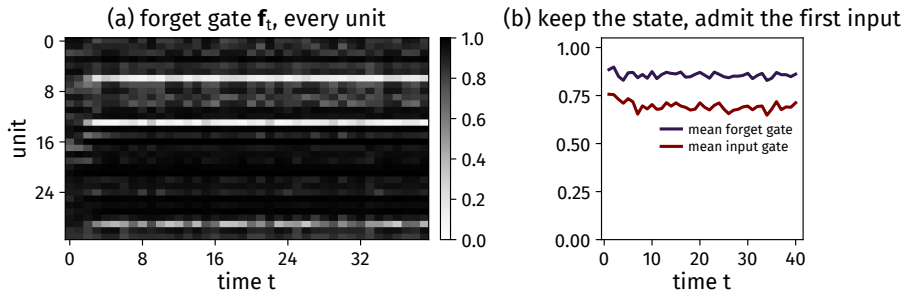
Proposition 5 (The cell's gradient is a product of forget gates)

Holding the gates fixed, along the cell's own path $\partial \mathbf{c}_t / \partial \mathbf{c}_k = \prod_{s=k+1}^t \text{diag}(\mathbf{f}_s)$, a product of numbers in $(0, 1)$ with no weight matrix and no activation derivative in it: with $\mathbf{f}_s \approx \mathbf{1}$ the gradient passes unattenuated.

- Proof. $\mathbf{c}_s = \mathbf{f}_s \odot \mathbf{c}_{s-1} + \mathbf{j}_s$ is linear in $\mathbf{c}_{s-1}$ with coefficient $\mathbf{f}_s$; chain the steps. □
- Against Proposition 2: the simple cell's factor $\text{diag}(g')$ U is typically well below one; the LSTM's is a gate the network can hold at one — the constant error carousel. The gates depend on $\mathbf{h}_{t-1}$ too, so there are other paths; this is the one that survives.

Hochreiter & Schmidhuber (1997); Gers et al. (2000); Goodfellow et al. (2016), §10.10.1.

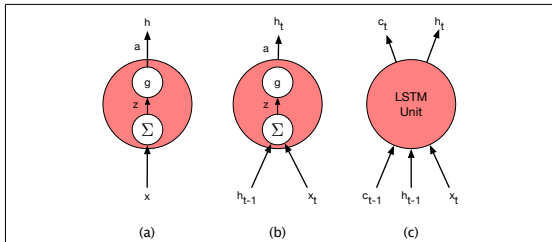
What the gates do while remembering



The LSTM trained here, on one length-40 sequence whose first symbol it must recall. (a) the forget gate of every unit against time: most units hold near one throughout, a few open and close with the input. (b) the mean forget gate stays at 0.86; the mean input gate is highest at the first step, when the fact arrives.

Jurafsky & Martin (2026), §14.5; Goodfellow et al. (2016), §10.10.1.

Gated units are drop-in modules



- (a) feedforward, (b) simple recurrent, (c) LSTM: the complexity is **inside**; from outside, only the cell vector is new
- So an LSTM drops into every architecture of Classes 27 and 29 and is unrolled and trained the same way

Jurafsky & Martin (2026), §14.5.1, Fig. 14.14.

Other Gated Cells

The gated recurrent unit

Definition 6 (GRU)

One gate decides both what to forget and what to write:

$$\mathbf{u}_t = \sigma(\mathbf{U}_u \mathbf{h}_{t-1} + \mathbf{W}_u \mathbf{x}_t) \quad \text{update gate}$$

$$\mathbf{r}_t = \sigma(\mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{W}_r \mathbf{x}_t) \quad \text{reset gate}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{W}_h \mathbf{x}_t) \quad \text{candidate}$$

$$\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t \quad \text{new state}$$

- No separate cell, three weight pairs against four. The update gate is a conditional leaky integrator per dimension; the reset gate chooses what of the past the candidate may see

Goodfellow et al. (2016), Eqs. 10.45–10.47; Cho et al. (2014); Chung et al. (2014).

The GRU's state stays between old and new

Proposition 7 (Convex update)

Since every entry of $\mathbf{u}_t$ lies in $(0, 1)$, each coordinate of $\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t$ lies between the old state and the candidate. Along the old state the derivative is $\text{diag}(\mathbf{u}_t)$, so a coordinate with $u \approx 1$ is copied unchanged and its gradient passes unchanged, exactly as on the LSTM's cell line.

- **Proof.** $u a + (1 - u) b$ with $u \in (0, 1)$ is a convex combination of a and b ; differentiate in a . $\square$
- Which pieces of the LSTM are needed? Ablations find no variant that beats both LSTM and GRU across tasks; the forget gate is the crucial ingredient, and a forget bias of 1 makes the LSTM as strong as the best variant.

Goodfellow et al. (2016), §10.10.2; Greff et al. (2017); Jozefowicz et al. (2015).

The GRU's state stays between old and new

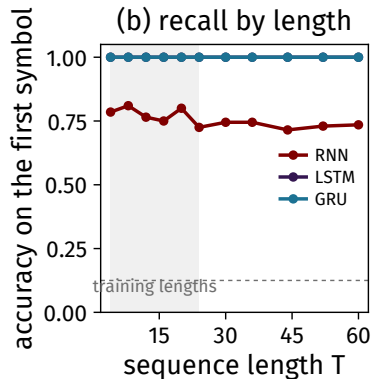
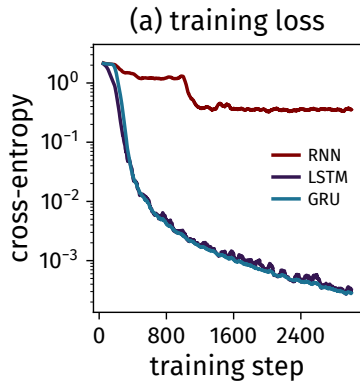
Proposition 7 (Convex update)

Since every entry of $\mathbf{u}_t$ lies in $(0, 1)$, each coordinate of $\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t$ lies between the old state and the candidate. Along the old state the derivative is $\text{diag}(\mathbf{u}_t)$, so a coordinate with $u \approx 1$ is copied unchanged and its gradient passes unchanged, exactly as on the LSTM's cell line.

- Proof. $u a + (1 - u) b$ with $u \in (0, 1)$ is a convex combination of a and b ; differentiate in a . $\square$
- Which pieces of the LSTM are needed? Ablations find no variant that beats both LSTM and GRU across tasks; the forget gate is the crucial ingredient, and a forget bias of 1 makes the LSTM as strong as the best variant.

Goodfellow et al. (2016), §10.10.2; Greff et al. (2017); Jozefowicz et al. (2015).

Three cells on one task



An RNN, an LSTM and a GRU of width 32, trained the same way for 3,000 steps to read 4–24 symbols and emit the first. (a) training loss. (b) accuracy by sequence length, tested to 60: the gated cells are perfect at every length, including far beyond what they trained on; the simple RNN never passes 81%, even at length 8. Chance is 1/8. Jurafsky & Martin (2026), §14.5; Goodfellow et al. (2016), §10.10.

Optimising Through Time

Clipping the gradient

Proposition 8 (Norm clipping keeps the direction)

Replace the gradient $\mathbf{g}$ by

$$\mathbf{g} \leftarrow \frac{\mathbf{g} v}{\|\mathbf{g}\|} \quad \text{if } \|\mathbf{g}\| > v.$$

The step is still along $\mathbf{g}$, a descent direction, and its length is at most ηv whatever the gradient's size.

- **Proof. A positive scaling changes length only, and $\|\eta \mathbf{g} v / \|\mathbf{g}\|\| = \eta v$.** □
- Why: the loss over many steps has **cliffs**, flat regions separated by walls exponentially steep in the number of steps, since $\mathbf{U}$ multiplies itself once per step; one step at a cliff can undo all the descent so far. Clipping addresses explosion only.

Goodfellow et al. (2016), §10.11.1, Eqs. 10.48–10.49; Pascanu et al. (2013); Mikolov (2012).

Clipping the gradient

Proposition 8 (Norm clipping keeps the direction)

Replace the gradient $\mathbf{g}$ by

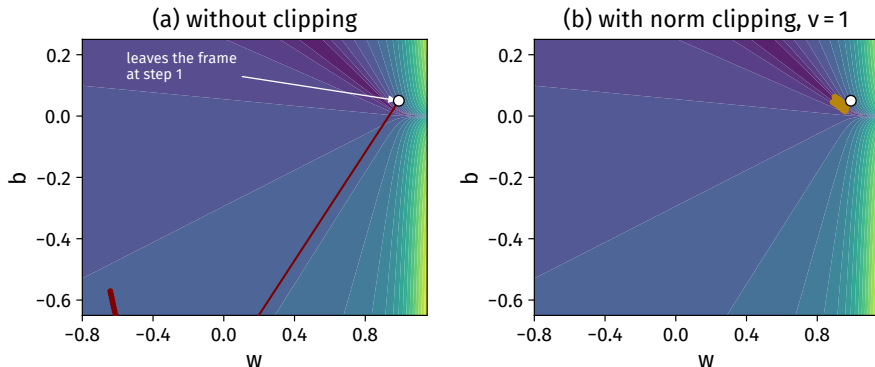
$$\mathbf{g} \leftarrow \frac{\mathbf{g} v}{\|\mathbf{g}\|} \quad \text{if } \|\mathbf{g}\| > v.$$

The step is still along $\mathbf{g}$, a descent direction, and its length is at most ηv whatever the gradient's size.

- Proof. A positive scaling changes length only, and $\|\eta \mathbf{g} v / \|\mathbf{g}\|\| = \eta v$. □
- Why: the loss over many steps has cliffs, flat regions separated by walls exponentially steep in the number of steps, since U multiplies itself once per step; one step at a cliff can undo all the descent so far. Clipping addresses explosion only.

Goodfellow et al. (2016), §10.11.1, Eqs. 10.48–10.49; Pascanu et al. (2013); Mikolov (2012).

Clipping at a cliff, run



A recurrent net with two parameters, $h_t = w h_{t-1} + b$ for 50 steps and a squared error at the end, so the loss has a cliff at $w = 1$. Gradient descent from the same start with the same step size: (a) without clipping the first step at the cliff throws the parameters across the plane, to a loss of 0.36; (b) with the norm clipped at $v = 1$ the steps are bounded and settle at a loss of 0.005. Goodfellow et al. (2016), Fig. 10.17, after Pascanu et al. (2013).

Encouraging the gradient to flow

- Clipping handles explosion; vanishing needs a path whose product of derivatives is near one — the gates — or a **regulariser** that asks for it
- Pascanu et al. penalise, at every step, how much the back-propagated gradient shrinks in crossing that step:

$$\Omega = \sum_t \left(\frac{\|(\nabla_{\mathbf{h}_t} L)^\top \partial \mathbf{h}_t / \partial \mathbf{h}_{t-1}\|}{\|\nabla_{\mathbf{h}_t} L\|} - 1 \right)^2,$$

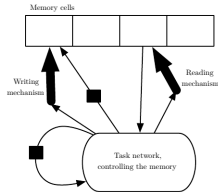
treating $\nabla_{\mathbf{h}_t} L$ as a constant for the purpose of the regulariser

- With clipping alongside it, this lengthens the dependencies a simple RNN can learn; with abundant data it is not as effective as an LSTM, which is why the gate won

Goodfellow et al. (2016), §10.11.2, Eqs. 10.50–10.52; Pascanu et al. (2013).

Where the gate leads: explicit memory

- The LSTM cell is a memory of one vector with learned write and read gates; a **memory network** is a set of such cells, addressed by content
- A task network writes and reads through soft attention over the cells — Class 27's softmax-weighted average
- Reading is differentiable, so it trains end to end; a fact is stored in one write, not many gradient steps



A network with an explicit memory: the task network controls the writing and reading mechanisms. Goodfellow et al. (2016), Fig. 10.18.

Goodfellow et al. (2016), §10.12; Weston et al. (2014); Graves et al. (2014).

Putting It Together

The cell, opened

	state	path across time	derivative along it
simple RNN	$\mathbf{h}_t$	through $\mathbf{U}$ and g every step	$\text{diag}(g') \mathbf{U}$, shrinks or explodes
leaky unit	μ_t	a linear self-connection	the constant α
LSTM	$\mathbf{h}_t$ and $\mathbf{c}_t$	the cell line	$\text{diag}(\mathbf{f}_t)$, chosen by the network
GRU	$\mathbf{h}_t$	the update gate's copy	$\text{diag}(\mathbf{u}_t)$, chosen by the network

Every fix is the same fix: give the gradient a path whose factors stay near one. The gate is the version where the network decides, at every step, which facts get that path.

Takeaways

1. A repeated linear map raises its eigenvalues to the power of the time: components decay or explode geometrically (Theorem 1), and the same product bounds the gradient of the nonlinear cell (Proposition 2)
2. A linear self-connection of weight near one is a running average with a long time constant, and a path the gradient survives (Proposition 3)
3. The LSTM splits the state into a cell and a hidden vector, and manages the cell with three gates (Definition 4); along the cell line the gradient is a product of forget gates, with no weight matrix in it (Proposition 5)
4. The GRU does the same with two gates and no separate cell; its update is a convex combination (Definition 6, Proposition 7)
5. Measured: the gated cells recall a fact across 60 steps and carry the gradient with them; the simple RNN does neither
6. Norm clipping bounds the step without changing its direction (Proposition 8); an LSTM is a better answer to vanishing than any regulariser

Class 29 puts the cell to work: language models, taggers, classifiers, and what stacking and bidirectionality buy.

References i

1. D. Jurafsky & J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, August 2026. Ch. 14 §14.5 (Eqs. 14.20–14.27, Figs. 14.13–14.14), §14.5.1.
2. I. Goodfellow, Y. Bengio & A. Courville, *Deep Learning*, MIT Press, 2016. §10.7 (Eqs. 10.36–10.39, Fig. 10.15), §10.8, §10.9, §10.10 (Eqs. 10.40–10.47, Fig. 10.16), §10.11 (Eqs. 10.48–10.52, Fig. 10.17), §10.12 (Fig. 10.18).
3. S. Hochreiter & J. Schmidhuber, “Long short-term memory”, *Neural Computation* 9(8), 1997.
4. F. A. Gers, J. Schmidhuber & F. Cummins, “Learning to forget: continual prediction with LSTM”, *Neural Computation* 12(10), 2000.
5. Y. Bengio, P. Simard & P. Frasconi, “Learning long-term dependencies with gradient descent is difficult”, *IEEE Trans. Neural Networks* 5(2), 1994.
6. R. Pascanu, T. Mikolov & Y. Bengio, “On the difficulty of training recurrent neural networks”, ICML 2013.
7. K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation”, EMNLP 2014; J. Chung et al., “Empirical evaluation of gated recurrent neural networks on sequence modeling”, 2014.

References ii

8. K. Greff et al., “LSTM: a search space odyssey”, *IEEE TNNLS* 28(10), 2017; R. Jozefowicz, W. Zaremba & I. Sutskever, “An empirical exploration of recurrent network architectures”, ICML 2015.
9. M. C. Mozer, “Induction of multiscale temporal structure”, NeurIPS 1992; T. Lin et al., “Learning long-term dependencies in NARX recurrent neural networks”, *IEEE Trans. Neural Networks* 7(6), 1996.
10. H. Jaeger & H. Haas, “Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication”, *Science* 304, 2004.
11. T. Mikolov, *Statistical Language Models Based on Neural Networks*, PhD thesis, Brno University of Technology, 2012.
12. J. Weston, S. Chopra & A. Bordes, “Memory networks”, ICLR 2015 (arXiv 2014); A. Graves, G. Wayne & I. Danihelka, “Neural Turing machines”, arXiv:1410.5401, 2014.

Questions?